

İSG BİLDİRİM SİSTEMİ

Sprint 2 Raporu

Kimlik Doğrulama & RBAC Çekirdeği

Sprint:	Sprint 2 — Kimlik Doğrulama & RBAC Çekirdeği
Kilometre taşı:	M1 — Temel Altyapı & Kimlik Doğrulama
Zaman:	Gün 3–4 (17 iş günü planı)
Kapsam:	Üretilen dosyalar, ilişkiler ve topolojik ER haritası
Cobweb büyüklüğü:	25 düğüm / 28 kenar (kümülatif)
Belge:	Sprint 2/8

1. Sprint 2 Amacı ve Kapsamı

Güvenlik çekirdeğini kurmak: kayıt/giriş, JWT tabanlı oturum, granüler izin (RBAC) modeli ve iki global guard. Şema; User, Role, Permission, UserStatus ve Rank ile genişler. seed.ts süper admin, sistem rolleri ve birimleri oluşturur. İstemcide giriş, kayıt ve onay-bekleme akışı çalışır hale gelir.

2. Bu Sprintte Üretilen Dosyalar

Aşağıdaki tablo, bu sprintte oluşturulan her dosyayı (veya grup klasörünü) tekil olarak ele alır: hangi katmana ait olduğu, hangi modül/özellik hitap ettiği ve başlıca ilişkileri. Grup halindeki dosyalar, ilgili oldukları modül altında birlikte açıklanır.

Dosya / Grup	Katman	Modül / Özellik (ne işe yarar)	Başlıca İlişki
backend/src/auth/auth.service.ts	Backend	Auth modülü çekirdeği — register (PENDING), login (bcrypt + JWT), me (izinlerle profil), profil/şifre güncelleme, onaycılara bildirim.	PrismaService, JwtService, AuditService; User/Role modelleri.
backend/src/auth/auth.controller.ts	Backend	Auth uçları — /auth/register, /login, /me, /profile, /password.	AuthService'i çağırır; @CurrentUser kullanır.
backend/src/auth/dto/auth.dto.ts	Backend	Girdi doğrulama — Register/Login/UpdateProfile/ChangePassword DTO'ları (class-validator).	ValidationPipe tarafından uygulanır.
backend/src/auth/jwt.strategy.ts	Cross-cutting	Token doğrulama — geçerli token'da kullanıcıyı DB'den TAZE yükler, izin listesini üretir (rol değişikliği anında etkili).	JwtAuthGuard bunu kullanır; AuthUser üretir.
backend/src/common/guards/jwt-auth.guard.ts	Cross-cutting	Global kimlik guard'ı — public olmayan tüm uçlarda token zorunlu.	app.module APP_GUARD; jwt.strategy.
backend/src/common/guards/permissions.guard.ts	Cross-cutting	Global yetki guard'ı — @RequirePermissions ile izinleri karşılaştırır, süper admin kısa devre, VERIFIED kapısı.	permissions.decorator, AuthUser.
backend/src/common/decorators/permissions.decorator.ts	Cross-cutting	Yetki bildirimi — uçlara gereken izinleri metadata olarak iliş­tirir.	PermissionsGuard okur.
backend/src/common/decorators/current-user.decorator.ts	Cross-cutting	Oturum kullanıcısı — AuthUser'ı controller parametresine enjekte eder.	Tüm korumalı controller'lar.
backend/prisma/seed.ts	Veri Modeli	Tohumlama — süper admin, 4 sistem rolü (izin setleriyle), varsayılan birimler.	User, Role, Department modellerini yazar.
backend/prisma/schema.prisma (genişleme)	Veri Modeli	Kimlik modelleri — User, Role, Permission/UserStatus/Rank enum'ları eklenir.	auth.service, jwt.strategy, seed bunları kullanır.
client/src/api/client.ts	Frontend	HTTP köprüsü — axios instance, her isteğe Bearer token, 401'de oturum temizleme.	Tüm istemci sayfaları backend'e bununla erişir.
client/src/auth/AuthContext.tsx	Frontend	Oturum durumu — /me ile profil+izin tutar, login/logout/refresh sağlar.	api/client; App, Layout, korumalı sayfalar.
client/src/pages/LoginPage.tsx	Frontend	Giriş ekranı — kimlik doğrulama arayüzü.	AuthContext.login → /auth/login.
client/src/pages/RegisterPage.tsx	Frontend	Kayıt ekranı — yeni hesap (PENDING).	api/client → /auth/register.
client/src/pages/PendingPage.tsx	Frontend	Onay-bekleme ekranı — VERIFIED olmayan kullanıcı için.	AuthContext (status).

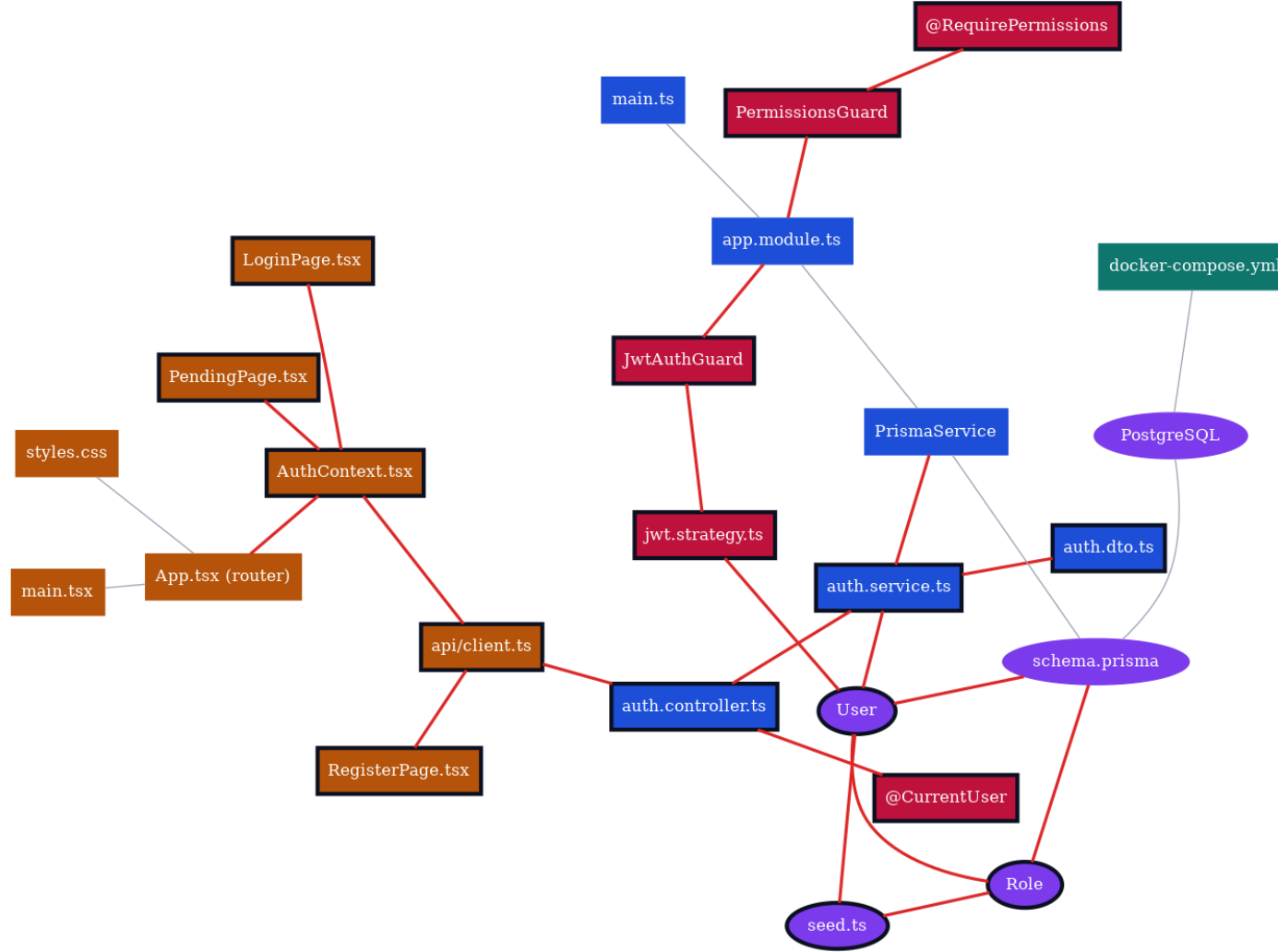
3. Dosya İlişkileri (Topolojik Anlatım)

Cobweb ilk kez iki katmanı birbirine bağlar. Backend'de güvenlik bir zincir oluşturur: istek → JwtAuthGuard → jwt.strategy (User'ı yükler) → PermissionsGuard (@RequirePermissions). auth.service User ve Role modelleriyle konuşur; seed bu modelleri doldurur. İstemcide api/client tüm HTTP trafiğinin tek kapısıdır; AuthContext bunun üstüne oturum durumunu kurar ve Login/Register/Pending sayfaları bu bağlama abone olur. Böylece grafikte ilk kez frontend (amber) ↔ backend (mavi) köprüsü (api/client → auth.controller) belirir.

4. Kümülatif Entity-Relation Haritası (Cobweb)

Aşağıdaki harita, projenin başından bu sprintin sonuna kadar (Sprint 1...2) oluşan TÜM dosya ve veri düğümlerini ve aralarındaki ilişkileri gösterir. Bu sprintte eklenen düğümler kalın koyu çerçeve, yeni ilişkiler kırmızı kenarlarla vurgulanmıştır. Harita her sprintte büyüyerek örümcek ağını (cobweb) yoğunlaştırır: bu sprint sonunda 25 düğüm ve 28 kenar içerir.

Sprint 2 - Kimlik Dogrulama & RBAC | Kumulatif Entity-Relation Cobweb (Sprint 1..2)



Şekil — Sprint 1...2 kümülatif entity-relation cobweb haritası (25 düğüm / 28 kenar). Renkler: teal=Altyapi, mor=Veri Modeli, mavi=Backend, amber=Frontend, kırmızı=Cross-cutting.

Renk lejanti: ● Altyapi ● Veri Modeli ● Backend ● Frontend ● Cross-cutting

5. Sprint Çıktısı & Kabul Kriterleri

Sprint sonunda aşağıdaki demo-edilebilir çıktılar elde edilir:

- Kullanıcı kayıt olabilir (PENDING) ve giriş yapabilir (JWT alır).
- Korumalı bir uca yetkisiz erişim 403 döner; onaysız kullanıcı uyarı alır.
- seed süper admin ile giriş yapıp /me doğru izinleri döndürür.

Bir sonraki adım: Kullanıcı / Rol / Birim Yönetimi (Sprint 3) — cobweb 37 düğüme büyür.